

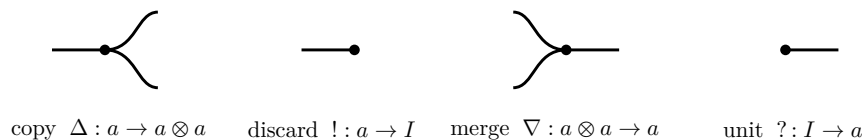
§2.124 in string diagrams — a proof from the axioms

Freyd & Scedrov, *Categories, Allegories* §2.124: $\text{Dom}(R \cap S) = 1 \cap SR^\circ$ — “a lemma we will use repeatedly”

An earlier note only **drew** the two sides and declared them equal. Here is the actual proof: the generators, the axioms they obey, two lemmas, and a step-by-step calculation in point-free (AOP) style, then drawn. Every algebraic step is tagged with the axiom it uses. The three underlying identities are proved in Lean for *arbitrary* relations — not a random sample — in `diag/S2_124.lean` (sorry-free, axioms `[propext]`).

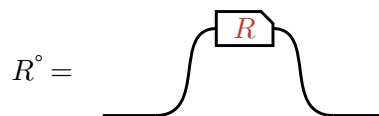
§1. What a string diagram is

An arrow of a symmetric monoidal category drawn as a circuit: white boxes are the **relations** $R, S : a \rightarrow b$; wires carry objects. Left-to-right juxtaposition is composition $;$ (the book’s $\times$ y), vertical stacking is the monoidal product $\otimes$ (the book’s $\oplus$). The identity 1 is a bare wire. An **allegory**’s operations are recovered from **one** extra structure on every object: a black-dotted **special Frobenius (co)monoid** — four generators.

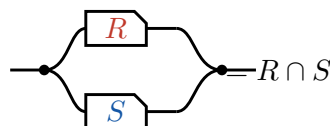


In REL these are the diagonal relations: **copy** is $x \mapsto (x, x)$, **discard** is $x \mapsto \checkmark$, **merge** = copy° , **unit** = discard° . Two derived pieces are used constantly — **converse** and **meet**:

Converse R° is **free**: bend both of R ’s wires around, so what was an input is read as an output. In REL, $x R^\circ y$ holds exactly when $y R x$.



Meet $R \cap S$ is **not** a new generator; it is the convolution $R \cap S = \Delta; (R \otimes S); \nabla$: copy the input, run R and S in parallel, then **merge**. The merge forces the two outputs to **coincide** — it demands both xRy and xSy .



The **domain** used below is $\text{Dom } R = 1 \cap R R^\circ$ (Freyd §2.122): the coreflexive $\sqsubseteq 1$ picking out the a that R relates to **something**.

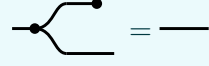
§2. The axioms of the calculus

Definition 2.1 (Special commutative Frobenius algebra (per object)).

The four dots obey these equations — the only moves a diagram may be rewritten by. σ is

the wire swap; **? is the unit** : a wire **created from nothing** ($? : \mathbf{I} \rightarrow \mathbf{a}$), the converse of discard **!**. The pair $(\Delta, !)$ copies and deletes; its converse pair $(\nabla, ?)$ merges and creates. Each law is drawn beside it (the monoid $(\nabla, ?)$ laws — assoc, unit, commutative — are the vertical mirrors of the first three):

counit $\Delta; (1 \otimes !) = 1$

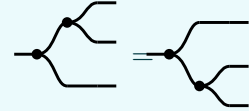


cocommutative $\Delta; \sigma = \Delta$



coassociative $\Delta; (\Delta \otimes 1) = \Delta; (1 \otimes \Delta)$

the picture re-brackets three wires for free; `delta_assoc` in the Lean spells the associator `tensAssoc` out

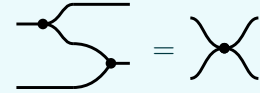


special $\Delta; \nabla = 1$ *copy-then-merge = wire*



Frobenius $(\Delta \otimes 1); (1 \otimes \nabla) = \nabla; \Delta = (1 \otimes \Delta); (\nabla \otimes 1)$

same suppression: `frob_left` / `frob_right` carry `tensAssoc` too, because the tensor is not strictly associative



Converse $(-)^{\circ}$ is the involution reversing composition and fixing the structure:

$$(P; Q)^{\circ} = Q^{\circ}; P^{\circ}, \quad P^{\circ\circ} = P, \quad \Delta^{\circ} = \nabla, \quad !^{\circ} = ?, \quad (P \otimes Q)^{\circ} = P^{\circ} \otimes Q^{\circ},$$

whence $(R \cap S)^{\circ} = R^{\circ} \cap S^{\circ}$. With `cup` = $?; \Delta$ and `cap` = $\nabla; !$, the special and Frobenius laws give the **yanking** (snake) equations, so a wire may be straightened.

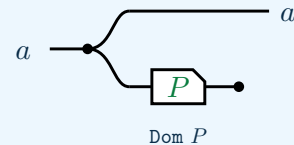
Derived — spider fusion. From the four groups above, any **connected** diagram built only from $\Delta, \nabla, !, ?$ with m inputs and n outputs equals the **single spider** $s_{(m,n)}$. This is the one power tool: a tangle of dots collapses to one node with the right number of legs.

§3. Two lemmas

Lemma 3.1 (domain = copy · run · discard).

$$\text{Dom } P = 1 \cap PP^{\circ} = \Delta; (1 \otimes P); (1 \otimes !).$$

Copy the input; run P on one copy; **discard** its output. The discard is the existential “ P relates to something”, so this is the domain coreflexive. (Both sides denote $\{(a, a) : \exists b. aPb\}$.)

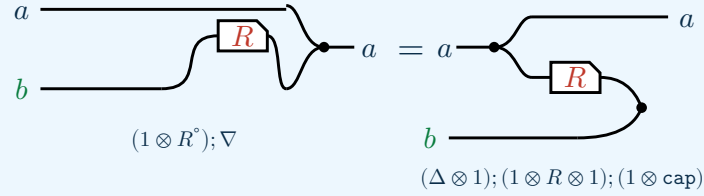


Lemma 3.2 (converse folds into a cap).

 For $R : a \rightarrow b$,

$$(1 \otimes R^\circ); \nabla = (\Delta \otimes 1); (1 \otimes R \otimes 1); (1 \otimes \text{cap}).$$

A converse R° feeding a merge equals: **copy** the surviving wire, run R forward, and **cap** its output against the incoming one — just “bend R , then yank” (converse + Frobenius). This is the move that turns $S R^\circ$ into a witness.



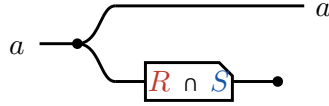
§4. The proof

Theorem 4.1.

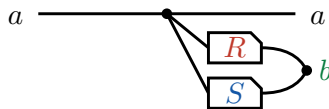
$$1 \cap SR^\circ = \text{Dom}(R \cap S).$$

Read the pictures top to bottom: $\text{Dom}(R \cap S)$ and $1 \cap S R^\circ$ both reduce to the **same** middle diagram W , so $\text{Dom}(R \cap S) = W = 1 \cap S R^\circ$. Each $=$ is one graphical move.

Dom(RnS) Lemma 1: copy a , run $R \cap S$ on one copy, discard

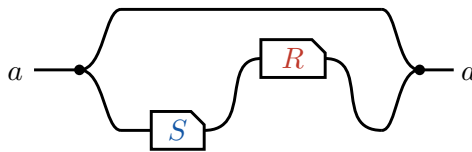


$=$ unfold $R \cap S = \Delta; (R \otimes S); \nabla$; $\nabla; ! = \text{cap}$; fuse the copies (coassoc)



the normal form $W = \Delta^3; (1 \otimes R \otimes S); (1 \otimes \text{cap})$ tap a ; run R, S cap to one b

$=$ Lemma 2: bend the R° straight



$1 \cap S R^\circ$ copy; the top wire is the 1 ; S then the bent R° ; merge

The one step with content is **Lemma 2** (bottom $=$): bending the R° in $S R^\circ$ straightens the S – R° loop into R beside S , both fed by the same tap, their b -outputs meeting at the

cap — the shared witness b . Everything else is unfolding η and spider fusion. No modular law appears; §2.124 is here **derived** from Frobenius + converse.

§5. Is this checked in Lean?

Yes — every boxed result above is a Lean theorem in `diag/S2_124.lean`, sorry-free, for arbitrary relations R, S :

in this note	in <code>diag/S2_124.lean</code>
Lemma 3.1 ($\text{domain} = \text{copy} \cdot \text{run} \cdot \text{discard}$)	<code>dom_cd</code>
Lemma 3.2 (converse folds into a cap)	<code>cv_merge</code>
Theorem 4.1 $1 \cap SR^\circ = \text{Dom}(RnS)$	<code>dom_inter_diag</code> — via <code>left_eq_W</code> , <code>right_eq_W</code>
the axioms of §2 (special, Frobenius, coassoc, ...)	the <code>CartBicat</code> fields of <code>diag/CB.lean</code> , and <code>special</code> there

How Lean checks them. Not in a concrete model: the statements are made over an arbitrary **cartesian bicategory of relations** (`diag/CB.lean`, Definition 4.1) and the proofs ARE the diagram rewriting above, step for step. Nothing is suppressed — the associators and unitors a picture hides are written out, which `diag/Monoidal.lean` supplies. `diag/RelSetCB.lean` then instantiates all of it at this repo's `Rel(Set)`.

The $1nPP^\circ \rightarrow \Delta; (1\otimes P); !$ collapse of Lemma 1 was, in an earlier version of this file, an extra relational axiom called `adequacy` ($\Delta; (R\otimes R); \nabla = R$). It is not an axiom: it is `meet_idem`, a theorem of Definition 4.1.

One genuine use of the modular law survives, inside Lemma 1 — $1 \cap P\tau \sqsubseteq PP^\circ$, which is `modular_right` at $S := \tau$. The modular law is itself a theorem here (`modular_of_frobenius`), derived from the Frobenius equation and the lax copy inequation, so the calculus is not assuming it.

The allegory-level §2.124, which Freyd proves from the modular law directly, is `dom_inter` in `Freyd/S2_10.lean`; its two halves are the two moves this diagram fuses into one:

<code>dom_inter</code> step (allegory / AOP)	string-diagram move
$1 \cap SR^\circ \sqsubseteq 1 \cap (RnS)R^\circ$ — <code>modular_le S R^\circ 1</code>	Lemma 2: bend R° , so R hangs beside S
$= 1 \cap R(R^\circ nS^\circ)$ — a coreflexive equals its reciprocal	read the tapped loop either way (converse of a coreflexive = itself)
$\sqsubseteq (RnS)(R^\circ nS^\circ) = \text{Dom}(RnS)$ — <code>modular_le R (R^\circ nS^\circ) 1</code>	coassoc: the two prongs share one tap Δ^3 and one cap
the outer 1 kept at each step (<code>le_inter</code> with $\sqsubseteq 1$)	the straight diagonal wire — the spider only taps it, never deletes it